IMDb Data Analysis with MongoDB

Course: Introduction to Big Data Analytics (7030ICT)

Tools: MongoDB, MongoDB Compass, JSON Schema validation, aggregation pipelines

Summary

I loaded the IMDb titles and ratings datasets (about 1.7 million title documents) into MongoDB, designed a JSON Schema for each collection, fixed the data quality problems that surfaced during import, profiled the data, and then used aggregation pipelines to answer business questions: movies released per year, and the top Fantasy and Adventure movies of 2021 by rating and by votes.

---

1. Setup

In Compass I created the database `imdb` with two collections, `titles` and `ratings`, and imported the light version of the dataset (`light_title_2.csv`, half the size of the original).

Collection	Source	Key fields
`titles`	titles file	`tconst`, `titleType`, `primaryTitle`, `originalTitle`, `isAdult`, `startYear`, `runtimeMinutes`, `genres`
`ratings`	ratings file	`tconst`, `averageRating`, `numVotes`

`tconst` is the unique title identifier and acts as the join key between the two collections.

---

2. Schema validation

Ratings

```
{
  $jsonSchema: {
    bsonType: "object",
    required: ["tconst", "averageRating"],
    properties: {
      tconst:        { bsonType: "string" },
      averageRating: { bsonType: "double" },
      numVotes:      { bsonType: "int" }
    }
  }
}
```

Titles: first version (strict, based on the field descriptions)

```
{
  $jsonSchema: {
    bsonType: "object",
    required: ["tconst", "primaryTitle"],
    properties: {
      tconst:         { bsonType: "string" },
      titleType:      { bsonType: "string" },
      primaryTitle:   { bsonType: "string" },
      originalTitle:  { bsonType: "string" },
      isAdult:        { bsonType: ["bool", "int"] },
      startYear:      { bsonType: "int" },
      runtimeMinutes: { bsonType: "int" },
      genres:         { bsonType: "string" }
    }
  }
}
```

The strict schema rejected a large share of the rows on import. Real-world data is messy, so I diagnosed each failure from the import error log and relaxed the schema step by step.

Import problems and how I fixed them

#	Problem	Cause	Fix
1	Rows rejected for `startYear` and `runtimeMinutes`	Missing values are stored as the literal text `\N`, which breaks the `int` rule	Allow `["int", "string"]` so `\N` passes without losing the real integers
2	Rows rejected for numeric titles (for example "99", "1900")	The import wizard inferred these titles as numbers	Allow `["string", "int", "double"]` on both title fields
3	Rows rejected for titles like "True" or "False"	The parser read them as booleans, and some titles as dates	Extend the allowed types to include `bool` and `date`
4	Rows rejected for empty titles	Empty cells import as `null`, and `primaryTitle` was required	Remove `primaryTitle` from `required` and allow `null`, so no records are lost

Titles: final version

```
{
  $jsonSchema: {
    bsonType: "object",
    required: ["tconst"],
    properties: {
      tconst:         { bsonType: "string" },
      titleType:      { bsonType: "string" },
      primaryTitle:   { bsonType: ["string", "int", "double", "bool", "date", "null"] },
      originalTitle:  { bsonType: ["string", "int", "double", "bool", "date", "null"] },
      isAdult:        { bsonType: ["bool", "int", "string"] },
      startYear:      { bsonType: ["int", "string"] },
      runtimeMinutes: { bsonType: ["int", "string"] },
      genres:         { bsonType: "string" }
    }
  }
}
```

---

3. Data profiling (Compass schema analysis)

Ratings

- `averageRating` is skewed toward above-average scores. Most values cluster between about 6.0 and 8.5, peaking around 7.0 to 7.5.
- `numVotes` has an extreme long tail. In the sample, the minimum is 5 votes and the maximum is 295,854, so a small number of titles attract most of the engagement.
- `tconst` is unique per document, which confirms it works as a clean key.

Titles

- `startYear` shows production growing sharply from the late 1990s, peaking at 2022, with records going back to 1896. Some values are the `\N` placeholder.
- `endYear` and `runtimeMinutes` are mostly `\N`. This reflects ongoing TV series and historical films with no recorded runtime.
- `isAdult` is heavily imbalanced: about 97% non-adult (0) and 3% adult (1).
- `titleType` is dominated by short-form content such as episodes and shorts, which far outnumber feature films.

- `genres` is a comma-separated string with a long tail of combinations, with Drama among the most common.
- Title fields occasionally contain numbers or nulls, which is why the schema had to be flexible.

---

4. Aggregation pipelines

These can be run in the Compass Aggregations tab (stage by stage) or in `mongosh`. The literal `\N` placeholder is written as `"\\N"`.

a) Movies released each year

```
db.titles.aggregate([
  { $match: { titleType: "movie", startYear: { $ne: "\\N" } } },  // movies with a valid year
  { $sortByCount: "$startYear" }                                   // group by year, count, sort
])
```

b) Top 5 Fantasy + Adventure movies of 2021 by rating

The genre must contain both Fantasy and Adventure, and may contain others.

```
db.titles.aggregate([
  { $match: {
      titleType: "movie",
      startYear: 2021,
      $and: [
        { genres: { $regex: "Fantasy",   $options: "i" } },
        { genres: { $regex: "Adventure", $options: "i" } }
      ]
  } },
  { $lookup: {                                  // join ratings on tconst
      from: "ratings",
      localField: "tconst",
      foreignField: "tconst",
      as: "ratingInfo"
  } },
  { $unwind: "$ratingInfo" },                   // drops titles with no rating
  { $sort: { "ratingInfo.averageRating": -1 } },
  { $limit: 5 },
  { $project: {
      _id: 0,
      primaryTitle: 1,
      startYear: 1,
      genres: 1,
      averageRating: "$ratingInfo.averageRating"
  } }
])
```

c) Top 5 Fantasy + Adventure movies of 2021 by number of votes

Same pipeline, ranked by `numVotes`.

```
db.titles.aggregate([
  { $match: {
      titleType: "movie",
      startYear: 2021,
      $and: [
        { genres: { $regex: "Fantasy",   $options: "i" } },
        { genres: { $regex: "Adventure", $options: "i" } }
      ]
  } },
  { $lookup: { from: "ratings", localField: "tconst", foreignField: "tconst", as: "ratingInfo" } },
  { $unwind: "$ratingInfo" },
  { $sort: { "ratingInfo.numVotes": -1 } },
  { $limit: 5 },
  { $project: {
      _id: 0,
      primaryTitle: 1,
      startYear: 1,
      genres: 1,
      numVotes: "$ratingInfo.numVotes"
  } }
])
```

Design notes

- `$match` runs first so only a small set of documents reaches the `$lookup`, which keeps the join fast on 1.7 million documents.
- `$unwind` turns the joined array into a single rating object per title.
- Ranking by average rating alone favours titles with very few votes, so in practice a minimum vote threshold (for example `numVotes >= 1000`) would give a more trustworthy top 5.

Skills demonstrated

MongoDB (Compass and mongosh), JSON Schema validation, data import troubleshooting, handling messy and missing data, schema profiling, aggregation pipelines (`$match`, `$sortByCount`, `$lookup`, `$unwind`, `$sort`, `$limit`, `$project`), and clear technical documentation.